Model Building Systems

DSC 102: Systems for Scalable Analytics

Haojian Jin

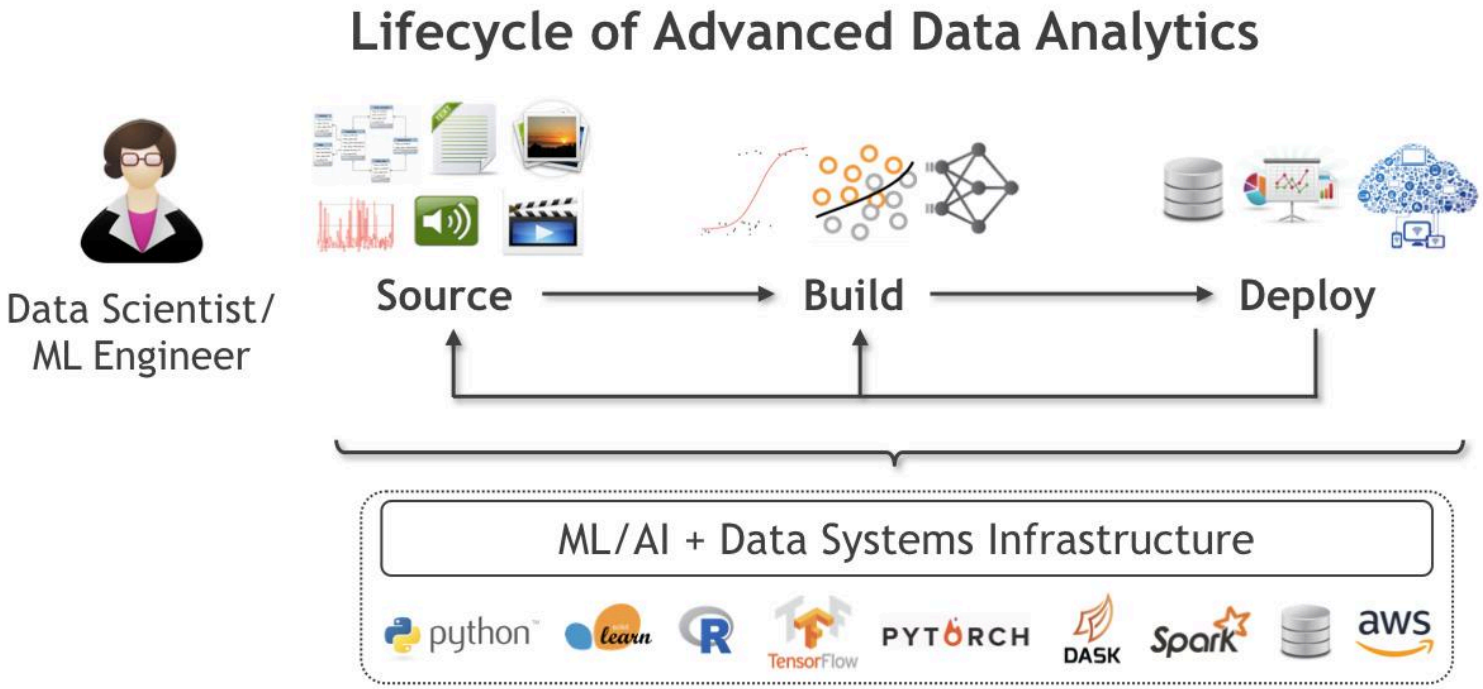
UC San Diego

Where we are

- **L4 to L6:** parallel and scalable data processing, built from the ground up: tasks, paged data, shards, BSP
- **L8:** the dataflow systems that automate it: MapReduce, Spark
- **Today:** the **build** stage of the ML lifecycle, where prepared data becomes prediction functions, and what it takes to scale it

Model building systems: feature engineering, model selection, hyper-parameter tuning, and scalable training and inference.

The lifecycle of ML-based analytics



Source: data acquisition and preparation · **Build:** feature engineering, training and inference, **model selection** · **Deploy:** serving, monitoring

Figure: ADALab, UC San Diego.

Four questions for today

1. What makes **model building** hard?
2. What is **feature engineering**, and how does each step **scale**?
3. How are models **selected and tuned**: algorithm selection, hyper-parameter tuning, AutoML?
4. How do **training and inference** systems scale?

Q1 · The building stage of the ML lifecycle

Model selection: convert prepared, ML-ready data into prediction function(s) and other analytics outputs.

What makes model building challenging and time-consuming?

- **Heterogeneity** of data sources, formats, and types
- **Configuration complexity** of ML models
- Large **scale** of data
- Long **training runtimes** of some models
- **Pareto optimization** over the application's criteria
- The data-generating process and the application **keep evolving**

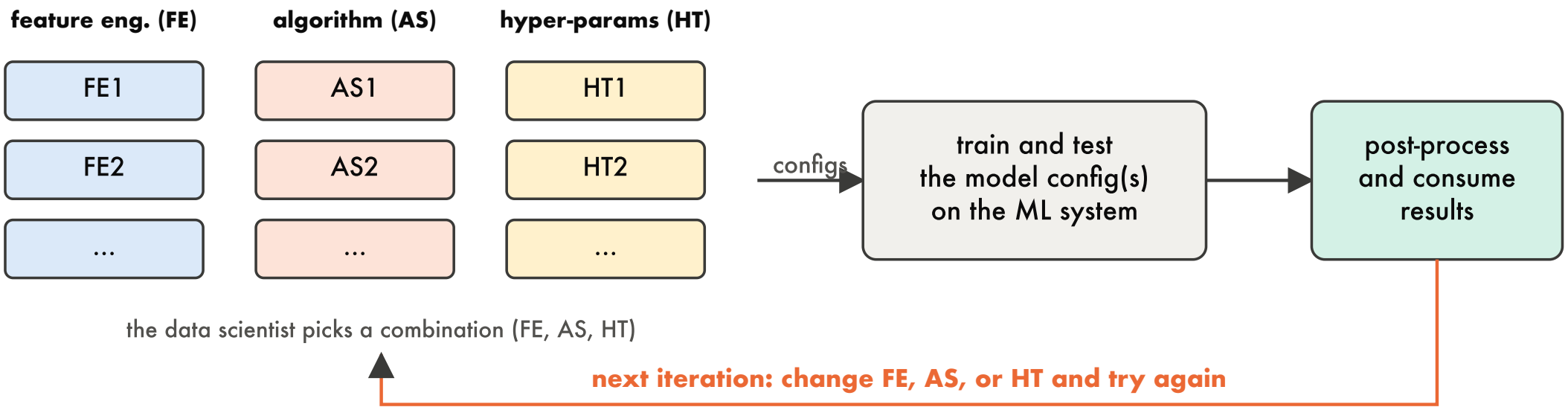
Three activities the data scientist steers

Each one invokes ML **training and inference** as subroutines:

Activity	The question it answers
Feature engineering (FE)	How to represent the signals appropriately for the domain of the prediction function?
Algorithm / architecture selection (AS)	What class of prediction functions, including the neural architecture, to use?
Hyper-parameter tuning (HT)	How to improve accuracy and other metrics by configuring the ML "knobs" better?

Kumar, McCann, Naughton, Patel, Model Selection Management Systems: The Next Frontier of Advanced Analytics, SIGMOD Record 2015.

The model selection process



a human steers the loop; AutoML automates part of it, usually HT first, then FE, rarely AS

Model selection is an iterative, exploratory process; the human decides on FE, AS, and HT, and each decision costs training runs.

What guides the decisions

Decisions on FE, AS, and HT are guided by many constraints and metrics:

- Prediction **accuracy**
- Data and feature **types**
- **Interpretability**
- **Tool** availability
- **Scalability** and **runtimes**
- **Fairness**, legal and regulatory issues

Decisions are typically **application-specific and dataset-specific**: recall the Pareto surfaces and tradeoffs.

There is no best model, only a best model for this data, this metric, and this budget.

Q2 · Feature engineering

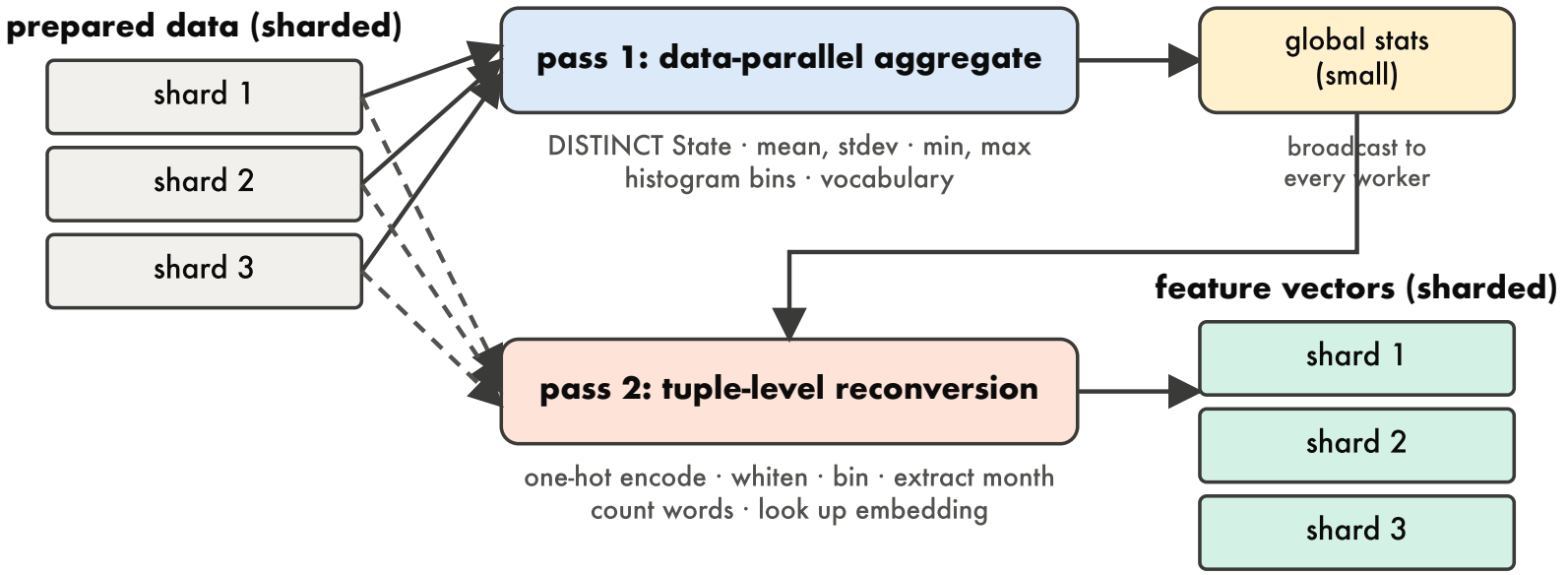
Converting prepared data into a **feature vector** representation for ML training and inference. Also called feature extraction, representation extraction.

An umbrella term for many tasks, depending on the type of model trained:

1. Recoding and value conversions
2. Joins and/or aggregates
3. Feature interactions
4. Feature selection
5. Dimensionality reduction
6. Temporal feature extraction
7. Textual feature extraction and embeddings
8. Learned feature extraction in deep learning

For each one, ask the systems question: what has to be computed over the whole dataset, and what is per tuple?

The scaling pattern behind most feature engineering



pass 1 is a GROUP BY or an algebraic aggregate from L6

pass 2 is a map-only job from L8: no shuffle, no Reduce

Two passes: a small set of global statistics, then a tuple-level function. You already know how to scale both.

1. Recoding and value conversions

Common on relational and tabular data: some **global column stats** plus code to **reconvert each tuple**.

UserID	State	Date	Upvotes	Comment	Label
143	CA	4/3/19	1539	"This restaurant is overrated"	–
337	NY	11/7/19	5020	"Not too bad!"	+

Why	Global stats (pass 1)	Reconversion (pass 2)
One-hot encoding: trees take categoricals, GLMs need numeric 0/1 vectors	`SELECT DISTINCT State`	Look up the domain hash table
Whitening: GLMs and ANNs want centered, scaled numerics	Mean and stdev (dense), or max and min (sparse)	Subtract and divide
Binning: Bayesian networks need discretized numerics	Histogram of the column	Map the number to a bin ID

Q: How do you scale each pass-1 statistic? (L6.)

2. Joins and aggregates

Most real-world relational datasets are **multi-table**: key–foreign-key joins, aggregation-then-join, and so on.

UserID	State	...	Label
143	CA	...	–
337	NY	...	+
143	CA	...	+

UserID	Age	Name
304	40	...
143	33	...

Example 1: join the reviews with the users on UserID; concatenate the user's info as extra features. What kind of join is this? How would you scale it?

Example 2: join the review table with **itself** on UserID to count the reviews and average the upvotes per user into a temporary table, then join that back. What kind of computation is this? (L6: GROUP BY, then a join.)

3. Feature interactions

Sometimes used on relational and tabular data, especially for **high-bias** models such as GLMs. Pairwise is common; ternary is not unheard of.

F1	F2	F3	Label
3	2	…	–
4	20	…	+
5	10	…	+

F1	F2	F3	F11	F12	F13	F22	…	Label
3	2	…	9	6	…	4	…	–
4	20	…	16	80	…	400	…	+
5	10	…	25	50	…	100	…	+

- **No global stats:** just a tuple-level function, so pass 2 alone; the feature count grows quadratically, which is the real cost
- Less popular since kernel SVMs, but **factorization machines** still need it

4. Feature selection

Basic idea: instead of using the whole feature set, use a **subset**.

```
{UserID, State, Date, Upvotes, Comment}  →  {State, Upvotes, Comment}  →  {Upvotes, Comment}  →  …
```

- Formulated as a **discrete optimization** problem; **NP-hard** in the number of features in general
- Many heuristics in ML and data mining, typically relying on information-theoretic criteria
- Typically scaled as an **outer loop** over training and inference: each candidate subset costs a training run
- Some ML users prefer a human-in-the-loop approach

5. Dimensionality reduction

Basic idea: transform the features into a different, **latent** space. Examples: PCA, SVD, LDA, matrix factorization.

```
(UserID, State, Date, Upvotes, Comment)  →  (F1, F2, F3) = (0.3, 4.2, -29.2)
```

Q: How is this different from feature selection?

- Feature selection **preserves the semantics** of each feature; dimensionality reduction typically does not, it combines features in "nonsensical" ways
- Scaling this is **non-trivial**: the global statistic is now a matrix decomposition, which is as hard as scaling a training algorithm (recall the Gramian and matrix algebra in L5)

6. Temporal feature extraction

Many relational and tabular datasets have a time or date column.

UserID	State	Date	Upvotes	Comment	Label
143	CA	**4/3/19**	1539	"This restaurant is overrated"	–
337	NY	**11/7/19**	5020	"Not too bad!"	+

- Most classifiers cannot use Date directly; extract **month** (categorical), **year**, **day of week**, elapsed time since an event, and so on
- **Reconversion:** a tuple-level function that extracts numbers and categories
- Sometimes global stats are needed to **calibrate** time (the first date, the range)
- Complex temporal features are the subject of time-series mining

7. Textual feature extraction

Many tabular datasets have text columns; in NLP the whole example is often just text. Most classifiers cannot process strings directly.

Comment	Label
"This restaurant sucks"	−
"Good good!"	+
"Pretty rad"	+

sucks	good	...	Label
1	0	...	−
0	2	...	+
0	0	...	+

- **Bag of words:** count each vocabulary word per comment; the **vocabulary** is the global stat, the counting is tuple-level
- **Knowledge bases:** entity dictionaries (celebrities, chemicals) add domain-specific features
- **Embeddings:** a numeric vector per token or sentence, trained offline on a large corpus (Word2Vec, GloVe, today LLM embedding APIs); at scale a **tuple-level lookup or call**

8. Learned feature extraction in deep learning

- A big win of deep learning: **no manual feature engineering** on unstructured data (NB: deep learning is still not the default on structured, tabular data)
- Very versatile, almost any data type as input or output:
  - **CNNs** over images, video, time series
  - **Transformers** and RNNs over text and sequences
  - **GNNs** over graph-structured data
- The neural **architecture** specifies how features are extracted and transformed internally, with weights that are **learned**
- "**Software 2.0**": the buzzword for such learned feature-extraction programs, versus hand-crafted feature engineering

Karpathy, Software 2.0, 2017.

Q3 · Hyper-parameter tuning

Hyper-parameters: knobs of an ML model or training algorithm that control the **bias–variance tradeoff** in a dataset-specific way to make learning effective.

- GLMs: the L1 or L2 regularizer; all gradient methods: the **learning rate**; SGD: the **batch size**

HT is an **outer loop** around training and inference:

- Most common: **grid search**, a set of values per knob and their Cartesian product
- Also common: **random search**, a subsample of the grid
- AutoML heuristics: Bayesian optimization, HyperOpt, successive halving

Every grid point is a full training run: the "many models, same data" workload Cerebro parallelized in L6.

Algorithm selection in classical ML

Not much to say: the ML user typically picks models and algorithms based on **intuition** and experience in classical (non-deep) ML.

- **Best practice:** first train **simple models** (logistic regression) as baselines, then try complex models (gradient-boosted trees such as XGBoost)
- **Ensembles:** build diverse models and aggregate their predictions

The systems cost of algorithm selection is the same as tuning: each candidate is another training run over the same data.

Architecture selection in deep learning

More critical in deep learning: the neural architecture is the **inductive bias** in classical-ML parlance, and it controls feature learning and the bias–variance tradeoff.

- **Some applications:** many off-the-shelf **pre-trained** models to do *transfer learning* from, for example the Hugging Face model hub; fine-tune or just prompt
- **Other applications:** swap the pain of hand-crafted feature engineering for the pain of neural architecture engineering

In 2026, "which pre-trained model and how much of it to retrain" is the architecture selection question for most teams.

Automated model selection: AutoML

Q: Can we automate the whole model selection process?

It depends. HT and most of FE are already largely automated in practice; (neural) AS is often dictated by the application.

Pros

- Ease of use; lower human cost
- Easier to audit
- Improves ML accessibility

Cons

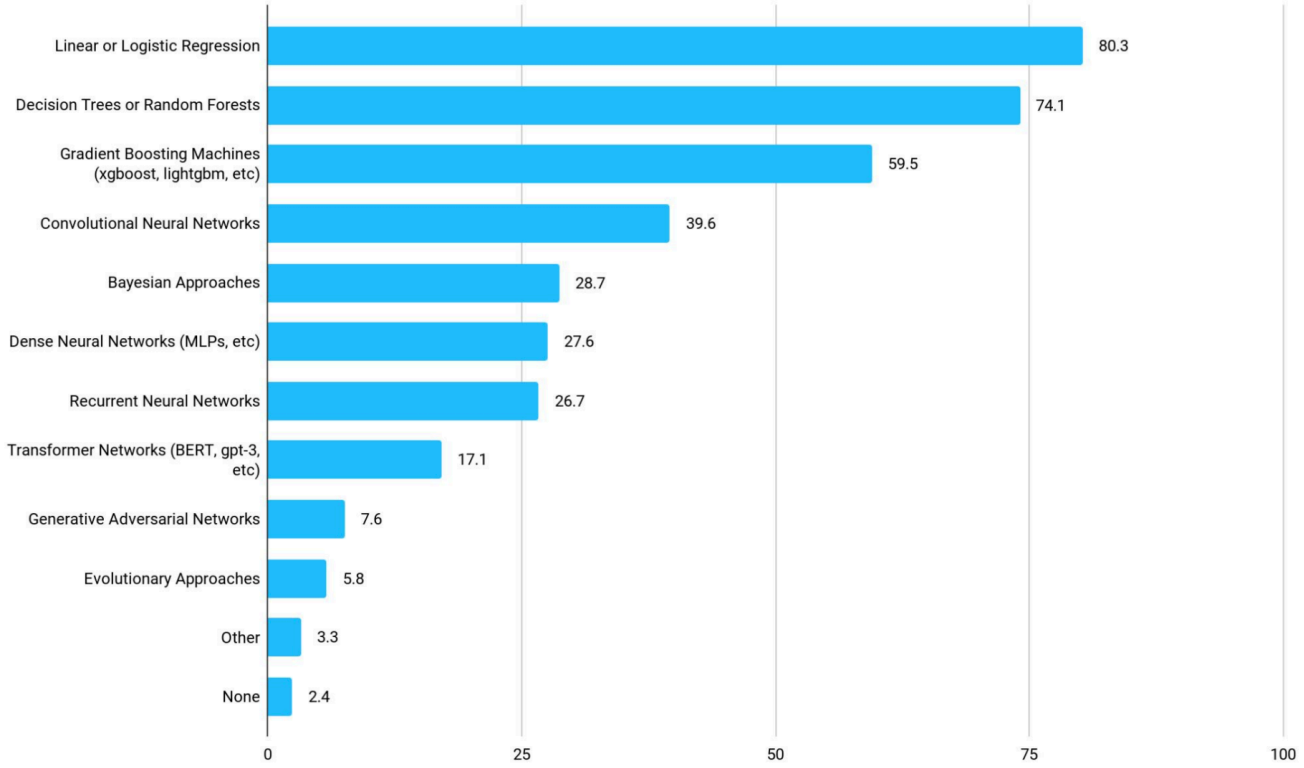
- Higher **resource** cost: many more training runs
- Less user control
- May waste domain knowledge

Pareto optima again; hybrids are possible. **But the data sourcing stage is still very hard to automate.**

Q4 · Major ML model families

Family	Examples	Inspired by
Generalized linear models (GLMs)	Linear and logistic regression	Statistics
Bayesian networks	Naive Bayes, graphical models	Causal reasoning
Decision-tree-based	CART, random forests, gradient-boosted trees	Symbolic logic
Support vector machines	Linear and kernel SVMs	Psychology (perceptrons)
Artificial neural networks	MLPs, CNNs, RNNs, Transformers	Brain neuroscience
Unsupervised	K-means, matrix factorization, LDA	Statistics, information theory

What people actually use: Kaggle 2021



- **Linear and logistic regression** lead at 80%
- **Tree learners** (forests, gradient boosting) next at 74% and 60%
- **Deep learning** (CNNs, MLPs, RNNs, Transformers) between 17% and 40%

Most model building is still tabular and classical, which is why the two-pass feature engineering pattern matters so much.

Kaggle, State of Data Science and Machine Learning 2021.

Scalable ML training systems

Scaling ML training is involved and **model-type-dependent**. Three orthogonal dimensions of categorization:

Dimension	One end	Other end
Scalability	In-memory library	Scalable ML system that works on larger-than-memory data
Target workloads	General ML library	Decision-tree-oriented, or deep-learning-oriented
Implementation reuse	Layered on top of a scalable data system (RDBMS, Spark)	Custom from-scratch framework

Take CSE 234 for more on scalable ML systems.

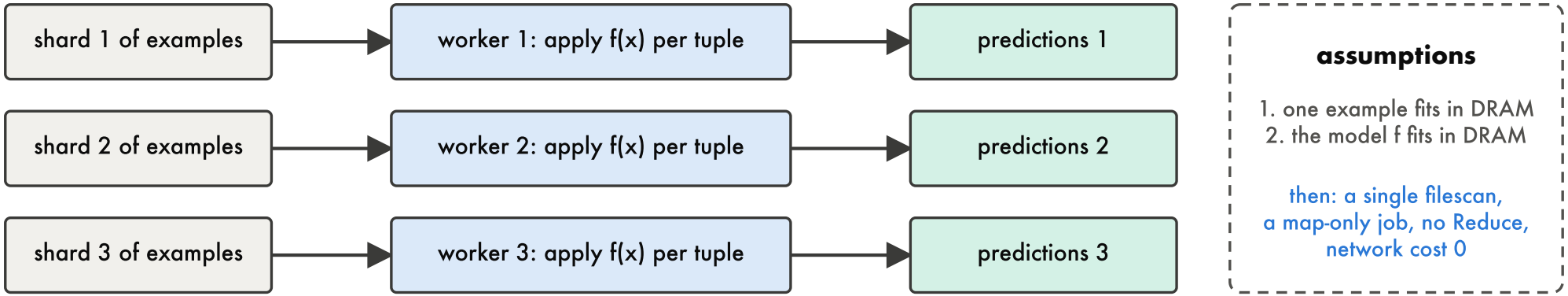
Major existing ML systems

Category	Examples
General, in-memory	scikit-learn, R packages, Weka
General, disk-based files	Vowpal Wabbit, XGBoost's external memory mode
General, layered on RDBMS or Spark	Apache MADlib, Spark MLlib, Dask-ML
Cloud-native	Amazon SageMaker, Google Vertex AI, Azure Machine Learning
AutoML platforms	H2O Driverless AI, DataRobot, AutoGluon
Decision-tree-oriented	XGBoost, LightGBM, CatBoost
Deep-learning-oriented	PyTorch, TensorFlow, JAX

Place each one on the three dimensions of the previous slide: that is the useful way to compare them.

Scalable ML inference

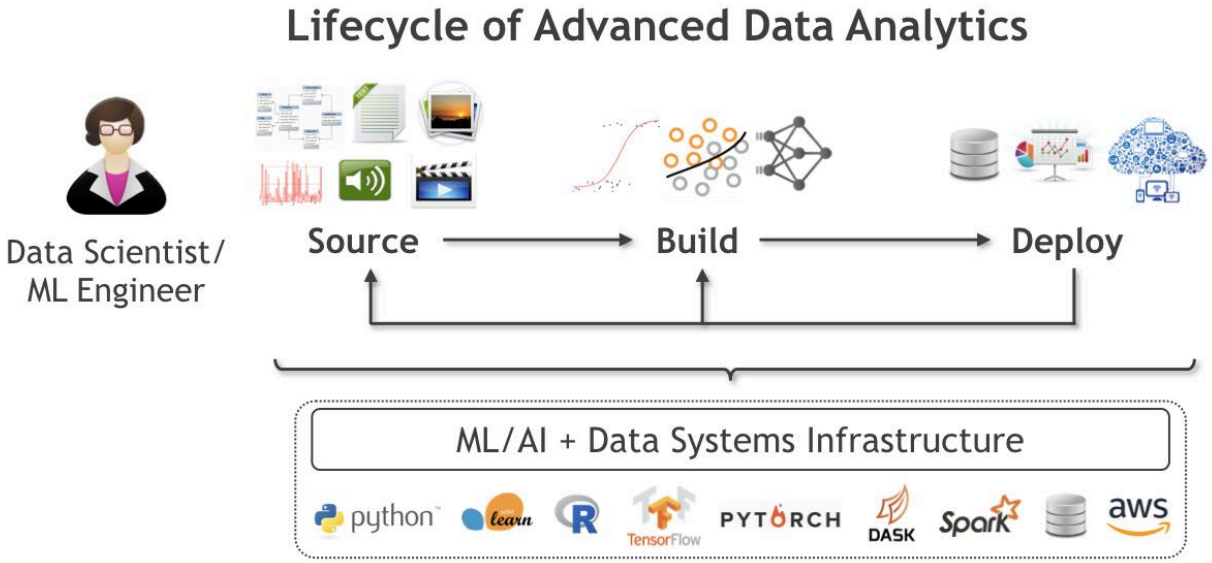
A trained model is just a **prediction function** f from a feature vector to a label or score.



if either assumption fails, f itself must be split and the access pattern gets model-specific

Q: How would you write this as a MapReduce job? (A map-only job: no shuffle, no Reduce.)

The lifecycle again



- **Source:** L5 and L6 scaled the data access; L8 the ETL
- **Build:** today, feature engineering and model selection on top of the same machinery
- **Deploy:** serving and monitoring, mostly beyond this course

Take CSE 234 for more on scalable ML systems.

What DSC 102 got you thinking about

The fundamentals of systems for scalable analytics.

- **"Systems":** what resources does a computer have? How do we store and efficiently compute over large data? What is the cloud, and what is an agent's memory?
- **"Scalability":** how do we scale and parallelize data-intensive computations: tasks, pages, shards, dataflow?
- **For "analytics":** sourcing data for ML; feature engineering and model selection systems; deploying ML models
- **Hands-on** experience with scalable analytics tools: Dask, Spark, the cloud

Review questions

1. What are the three activities of model selection, and what does each one invoke as a subroutine?
2. Describe the two-pass pattern for scaling feature engineering. Which L6 operation is pass 1? Which L8 job is pass 2?
3. Give one feature-engineering step that needs no global statistics, and one whose global statistic is a matrix.
4. What is the difference between feature selection and dimensionality reduction?
5. Why is hyper-parameter tuning an "outer loop", and what does that imply for its cost?
6. State the two assumptions under which inference is a single filescan. What happens when one fails?
7. Name one pro and one con of AutoML.

Key takeaways

1. **Model building is a loop:** pick feature engineering, algorithm, and hyper-parameters; train and test; iterate; the human steers, AutoML helps
2. **Most feature engineering scales as two passes:** small global statistics from a data-parallel aggregate, then a tuple-level map
3. **Tuning and selection are outer loops** that multiply training runs; that multiplication is the systems problem
4. **Training systems** differ along scalability, target workload, and implementation reuse
5. **Inference at scale is often just a filescan**, as long as the example and the model fit in memory